School of Computer Engineering

Manipal Institute of Technology, Manipal



Stylometric and Structural Classification Framework for Detecting LLM-Generated Code Submissions

Machine Learning Mini Project Evaluation

Academic year 2026–2027

Yash Raj Agarwal — 240905558

Nishaanth S — 240905422

Aman Sharma — 240905500

Faculty guide: [INSERT FACULTY GUIDE NAME]

CodeTrace AI • static Python classification prototype

Stylometric and Structural Classification Framework for Detecting LLM-Generated Code Submissions

Yash Raj Agarwal — 240905558; Nishaanth S — 240905422; Aman Sharma — 240905500

School of Computer Engineering
Manipal Institute of Technology, Manipal

Abstract - This project implements a local Python code-style classification pipeline as an undergraduate machine learning demonstration. Source code is inspected without execution using tokenization, regular expressions, and abstract syntax trees. Eighteen numerical features are combined with word unigram and bigram term-frequency inverse-document-frequency representations. Logistic Regression, Decision Tree, and Random Forest classifiers are trained and compared using group-disjoint training, validation, and test partitions. The demonstration dataset contains 160 unique labeled examples after removing 40 duplicate rows from 200 synthetic records. The split comprises 96 training, 32 validation, and 32 test examples. All three classifiers achieve accuracy, precision, recall, and F1 of 1.000 on the small held-out test partition; Logistic Regression is selected by validation F1 with an insertion-order tie break. These scores reflect repetitive synthetic style patterns rather than verified authorship. Controlled renaming and structural refactoring can change a predicted class while preserving program logic. A FastAPI service and React interface demonstrate saved-model inference, feature display, and evaluation visualization. The completed system supports reproducible teaching and exposes the need for provenance-verified datasets and stronger cross-task evaluation before research or educational decisions.

Index Terms - Abstract syntax trees; code stylometry; supervised classification; TF-IDF.

INTRODUCTION

Source code is constrained by syntax, shared algorithms, libraries, and assignment specifications. Similar solutions can therefore arise from different authors. Editing and refactoring further weaken any assumed relationship between surface style and origin. The project studies measurable lexical and structural signals through a small supervised learning pipeline. Academic integrity is the motivation, but source-code style alone cannot establish misconduct. Findings about general text detection [2] motivate caution; they are not treated as numerical evidence about code detectors. The contribution here is a transparent end-to-end implementation and reproducible sensitivity experiments, rather than a validated forensic detector.

LITERATURE SURVEY

Adegbite and Kotzé [1] investigate the directly relevant distinction using C# metric features and multiple classifiers. Burrows et al. [3] provide older source-code attribution context, and Breiman [4] supplies the ensemble foundation. These studies motivate baselines without implying that their results transfer to this project. The verified-source table accompanies this report. The requested 20–25 indexed journal articles could not be fully verified from accessible primary sources in this review. Four publisher-verified journal papers are included; only the directly relevant AJIC venue has explicit indexing evidence recorded here (Web of Science via SciELO Citation Index, not a claim of SCIE). Two foundational papers predate 2020. No conference paper or blog is substituted to inflate the qualifying count. This is a documented literature-coverage gap, not a systematic-review claim.

OBJECTIVES

Implement a reproducible static Python feature pipeline; compare three understandable classifiers under a group-disjoint split; expose predictions and real evaluation outputs through a local website; and test sensitivity to naming, comments, formatting, structure, hyperparameters, feature groups, thresholds, and an unseen task. The original synopsis proposed additional advanced models and consented classroom data. Those are future scope, not completed contributions.

DATA SOURCE

200 synthetic CSV rows; 40 duplicate rows removed; 160 unique examples (80 human-style, 80 AI-style); 20 synthetic problem groups; 96 training, 32 validation, 32 test samples. Both labels are synthetic style categories

written into a generator, not verified human or LLM provenance. The 20 group IDs vary a few filtering templates and do not represent 20 independent algorithmic tasks. Perfect scores describe template separation only. The generator assigns label strings human and ai to four templates in each category, with naming and modulo variations. Its case-sensitive replacement of "% 2" does not transform the compact "%2" variant, and some docstrings still describe even numbers after a modulus variation. Thus the generator is useful as a software fixture but is not a controlled authorship benchmark. No actual student submissions, LLM API outputs, DroidCollection, or LLM-AuthorBench data are imported. The importer drops missing code/label rows, strips boundaries, lowercases labels, removes invalid labels and blank code, rejects conflicting labels for identical code, and removes exact duplicates. Missing problem IDs receive row-specific fallback groups, which cannot protect against hidden shared tasks.

TABLE 1. ACTUAL SPLIT DISTRIBUTION.

Partition	Human	AI	Total
Train	48	48	96
Validation	16	16	32
Test	16	16	32

METHODOLOGY

The actual flow is code entry → user-selected Python → request validation → static feature extraction → saved transformation → classifier probability → threshold at 0.5 → display. Automatic language identification is absent. At training time the numerical features and source strings are separated from labels. GroupShuffleSplit with seeds 42 and 43 creates disjoint group partitions. Vectorizer vocabulary, inverse-document frequencies, scaler means, scaler variances, and classifier parameters are fitted only on training data. Validation selects the model; test metrics describe frozen fitted candidates. The final selected pipeline remains fitted on the 96 training examples; there is no train-plus-validation refit. Figure 1 shows the service architecture, and Table 1 reports the partitions.

IMPLEMENTATION DETAILS

backend/features.py contains 18 numerical features. Comment ratio now equals comment-token count divided by nonblank lines that are not comment-only, with denominator at least one. Identifier statistics count occurrences, not only unique names. The snake-style regular expression also accepts plain lowercase identifiers. Numeric token count excludes layout, end-marker and comment tokens but includes keywords, punctuation, and literals. AST node count includes contexts such as Load; nesting depth counts selected control constructs plus function definitions, not raw AST depth. Comprehension generators are not counted as ast.For loops. Tokenizer failures preserve any partial lexical counts; AST syntax failure leaves AST counts at zero internally, while the public API now rejects invalid syntax. These details matter when interpreting the signals.

ColumnTransformer combines TfidfVectorizer(max_features=1200, ngram_range=(1,2), sublinear_tf=True, token_pattern matching word tokens) with StandardScaler on all numeric features. Default TF-IDF lowercases tokens, smooths IDF and L2-normalizes each text vector. There is no separate supervised feature selector, dimensionality reduction, entropy estimator, or cyclomatic-complexity computation. The vocabulary cap is a frequency-based limit, not research-driven feature selection.

Logistic Regression uses C=1, balanced class weights, max_iter=1500 and seed 42. Decision Tree uses max_depth=12, min_samples_leaf=2, balanced weights, seed 42 and default Gini criterion. Random Forest uses 160 trees, min_samples_leaf=2, balanced weights, seed 42 and n_jobs=-1; other parameters use installed-library defaults, recorded in the inventory. Numeric scaling is shared for simplicity and is not required by axis-aligned trees. No grid search or cross-validation is implemented in the production trainer.

FastAPI exposes GET /api/health, /api/models, /api/metrics and POST /api/analyze. Pydantic bounds code length to 20–20,000 characters; language must be Python and syntax must parse. No submitted program executes. predict.py loads the joblib artifact on each request, computes numeric features, applies the fitted transformer and classifier, locates the ai class explicitly, and returns probability, predicted class, model name and descriptive features. joblib artifacts must be trusted local files. Only the selected model is serialized. The artifact now records schema and library versions plus dataset hash.

The React 18 TypeScript SPA uses local component state for Analyze, Model Performance and About views, with Tailwind 4 and custom CSS. Fetch sends JSON to localhost:8000/api. The metrics endpoint reads the actual JSON result file. Numeric signals shown for a sample are descriptive, not signed local attributions. Global feature importance is absolute Logistic Regression coefficients or impurity-based importance for trees. No SHAP, permutation explanation, account system, database or cloud deployment is present.

MATHEMATICAL MODEL

For an AI-oriented score $z=w^Tx+b$, $p(ai|x)=1/(1+\exp(-z))$ (1). Here w and b negate the stored binary parameters because the second sorted class is human. The training loss is regularized binary log loss with

class weighting; C controls inverse regularization strength. For a token t in document d, sublinear term frequency is 1+ln(count(t,d)) when count is positive. Smoothed inverse-document frequency is ln((1+N)/(1+df(t)))+1 (2). The TF-IDF vector is then normalized to unit L2 norm. Numeric scaling is $z_j=(x_j-\mu_j)/\sigma_j$ with statistics fitted on training examples (3). Decision trees minimize weighted child impurity using Gini $G=1-\Sigma_k p_k^2$ (4); the forest averages tree class probabilities. Prediction uses p(ai)≥0.5. The reported confidence is max(p(ai),1−p(ai)); it is uncalibrated. Precision=TP/(TP+FP), recall=TP/(TP+FN), F1=2PR/(P+R), accuracy=(TP+TN)/n (5), with ai as the positive label.

Results and Discussion

Table 2 compares actual held-out scores. All three models have accuracy=precision=recall=F1=1.000 on 32 synthetic test examples. The selected-model confusion matrix has TN=16, FP=0, FN=0 and TP=16 (Figure 2). Validation scores also tie; deterministic dictionary order selects Logistic Regression. The perfect result is unsurprising because the generator places docstrings in the AI-style templates and repeats forms across group boundaries. Group disjointness prevents identical problem IDs crossing splits, but does not remove template-family overlap. The selected global importance ranking begins with code length, docstring count and average identifier length; Figure 3 reports the actual stored values.

The controlled experiments use a fixed even-filter baseline. Renaming, comments and formatting preserve behavior; refactoring to a comprehension preserves behavior for ordinary finite lists of integers. Predictions and probabilities are reported in Table 3 from experiment_results.csv. A change of predicted class after renaming demonstrates model sensitivity to style, not altered authorship. Regularization and ablation are evaluated only on validation data to avoid repeated test-set tuning. Thresholds 0.4–0.7 leave validation scores unchanged on these highly separable examples. ROC AUC is newly computed in the experimental runner on validation probabilities; it is not a production dashboard metric. The factorial probe is an unlabeled out-of-distribution example, so one prediction cannot establish unseen-task accuracy. No test errors occur on this fixture; consequently real-world false-positive rates cannot be estimated. The experiment is reproducible but not statistically adequate for deployment.

TABLE 2. HELD-OUT SYNTHETIC TEST METRICS. LR: LOGISTIC REGRESSION; DT: DECISION TREE; RF: RANDOM FOREST.

Metric	LR	DT	RF
Accuracy	1.0	1.0	1.0
Precision	1.0	1.0	1.0
Recall	1.0	1.0	1.0
F1	1.0	1.0	1.0

TABLE 3. SAVED-MODEL INPUT SENSITIVITY. AI-CLASS PROBABILITIES, NOT PROVENANCE CERTAINTY.

Modification	Class	AI score
original	human	0.0448
variable naming	ai	0.9983
comment density	human	0.0863
formatting	human	0.0176
structure	ai	0.5442
unseen problem	human	0.0122

Conclusion and Future Scope

The implementation demonstrates a complete static-feature ML pipeline, saved preprocessing and inference, actual evaluation reporting, and a local web interface. It also demonstrates that equivalent code can receive very different probability estimates. Its completed contribution is instructional reproducibility. A meaningful research study requires consented and provenance-verified human data, matched generated outputs from known model versions, task/template/author-aware splits, repeated grouped evaluation, independent test sets, calibration, and uncertainty analysis. Additional languages, naming entropy, cyclomatic complexity, CodeBERT, XGBoost, richer AST representations and adversarial evaluation remain future scope. No authorship or academic-misconduct conclusion should be based on this system alone.

References

[1] A. A. Adegbite and E. Kotzé, “Detection of GenAI-produced and student-written C# code: A comparative study of classifier algorithms and code stylometry features,” The African Journal of Information and Communication, no. 35, pp. 1–20, 2025, doi: 10.23962/ajic.i35.21309.

[2] D. Weber-Wulff et al., “Testing of detection tools for AI-generated text,” International Journal for Educational Integrity, vol. 19, art. 26, 2023, doi: 10.1007/s40979-023-00146-z.

[3] S. Burrows, A. L. Uitdenbogerd, and A. Turpin, “Comparing techniques for authorship attribution of source code,” Software: Practice and Experience, vol. 44, no. 1, pp. 1–32, 2014, doi: 10.1002/spe.2146.

[4] L. Breiman, “Random Forests,” Machine Learning, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

SUPPORTING FIGURES AND LITERATURE COMPARISON

Figures 1–5 provide the architecture, measured results and an actual application capture referenced in the report. The one-column appendix keeps the evidence readable; it is an intentional academic addition to the conference template.

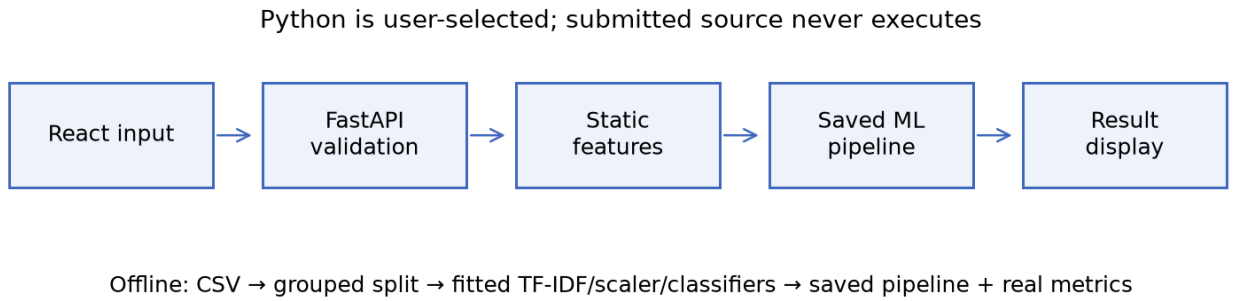


FIGURE 1. IMPLEMENTED INFERENCE PATH AND SEPARATE OFFLINE TRAINING.

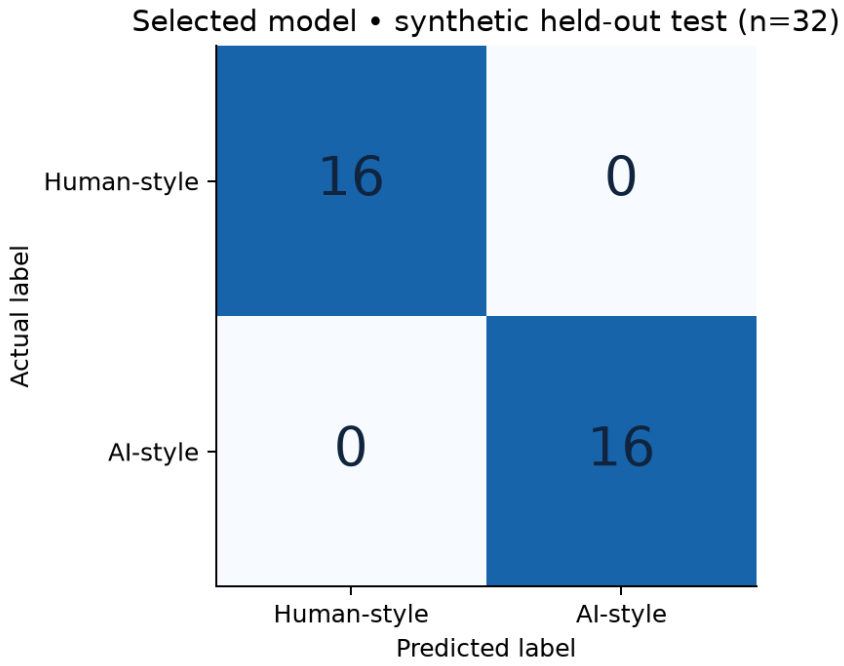


FIGURE 2. SELECTED-MODEL TEST COUNTS ON 32 SYNTHETIC EXAMPLES.

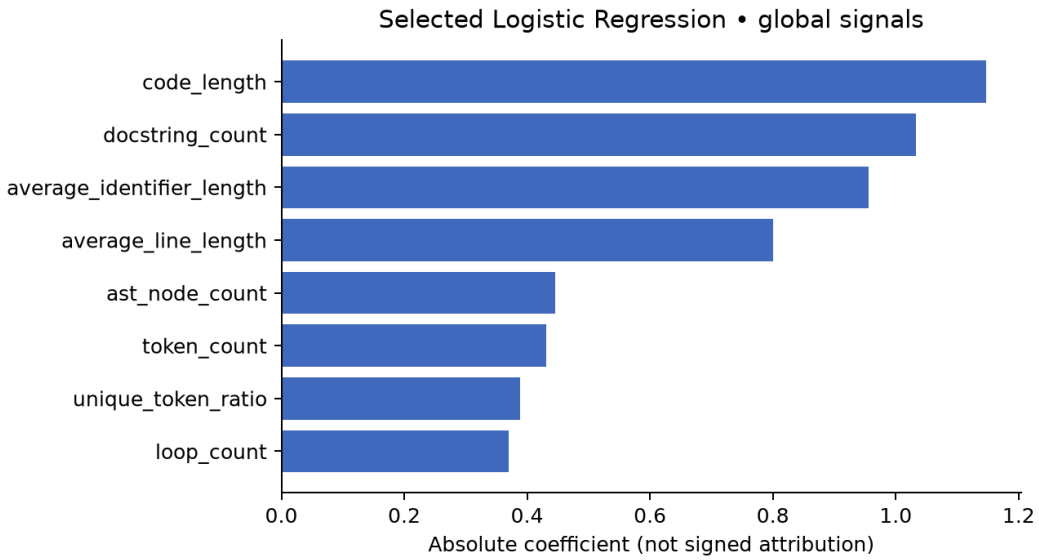


FIGURE 3. ABSOLUTE COEFFICIENTS; GLOBAL RANKING WITHOUT SIGNED LOCAL ATTRIBUTION.

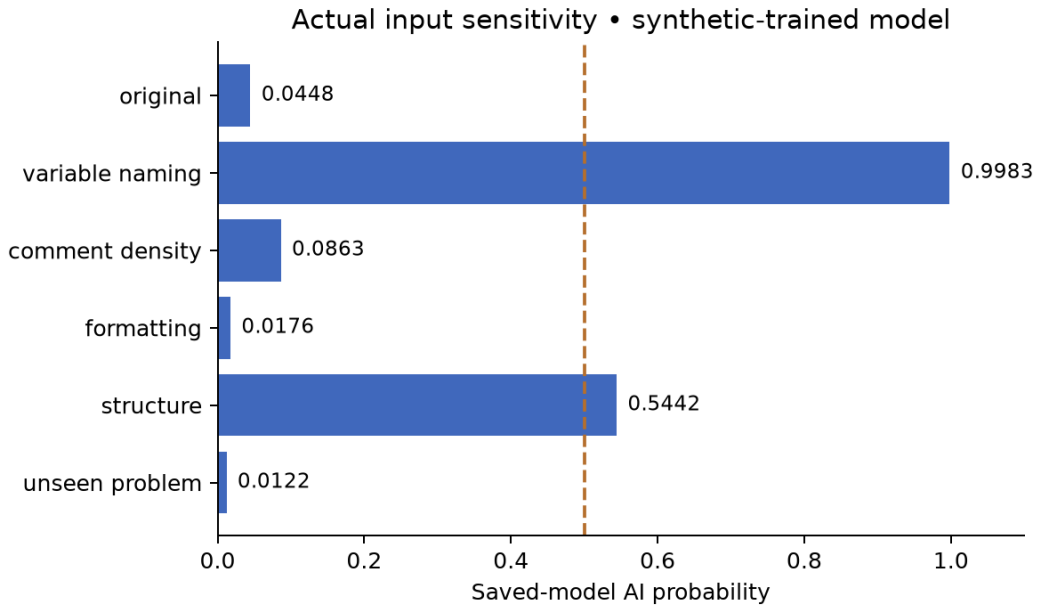


FIGURE 4. ACTUAL PROBABILITY CHANGES AFTER CONTROLLED SOURCE MODIFICATIONS.

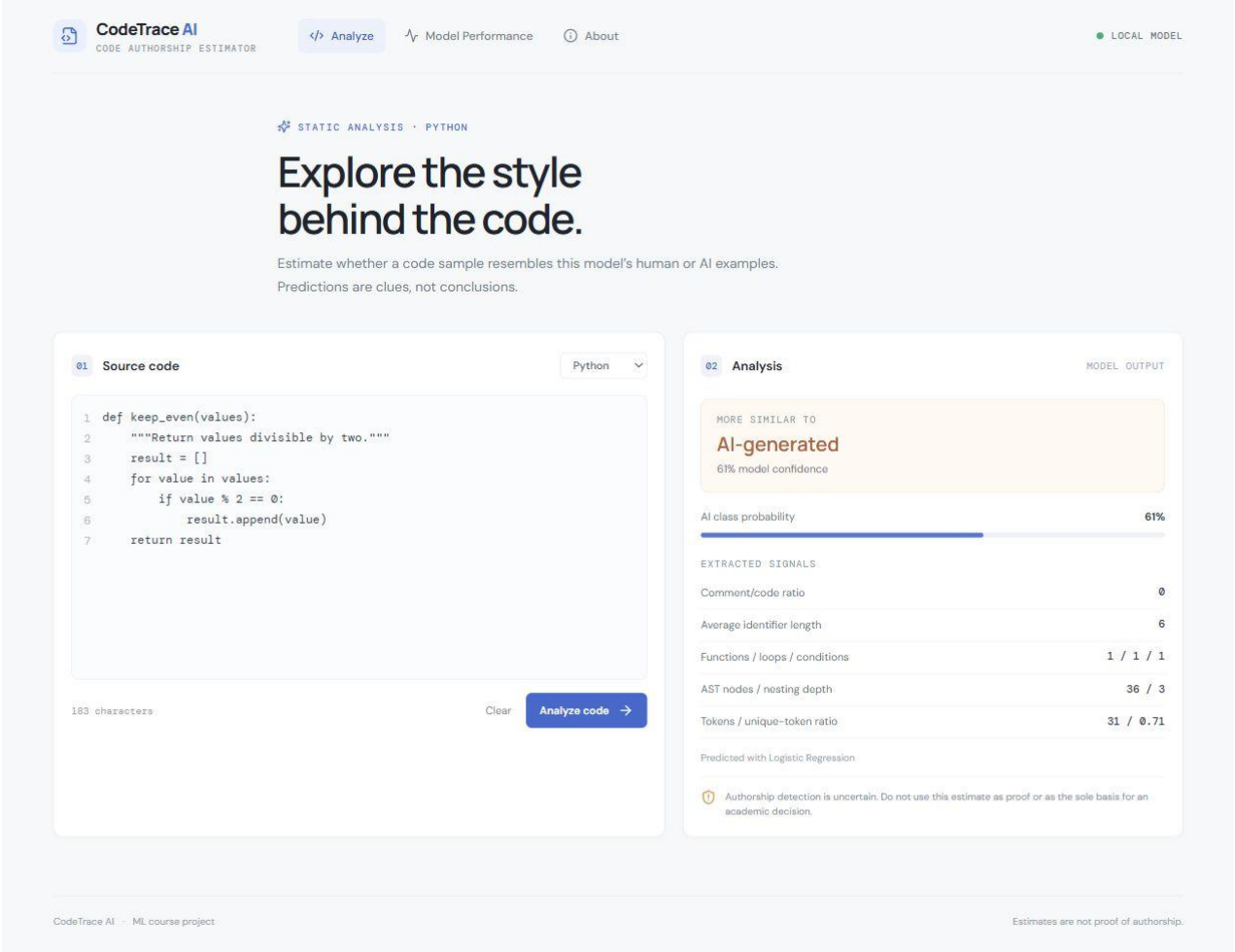


FIGURE 5. ACTUAL RUNNING APPLICATION WITH A SAVED LOGISTIC REGRESSION OUTPUT.

The interface output is an estimate. The raw feature panel is descriptive and is not a SHAP explanation. This screenshot was captured from localhost after a real API request.

TABLE 4. VERIFIED LITERATURE COMPARISON AND RELEVANCE.

Paper	Year	Methodology	Dataset	Key findings	Limitations	Relevance
[1] Detection of GenAI-produced and student-written C# code: A comparative study of classifier algorithms and code stylometry features,	2025	Code metrics and six classifiers	Verified student C# and matched GenAI samples	Code stylometry supported discrimination in the studied setting	Single educational/language setting; not transferable evidence for our synthetic Python data	Direct task relevance
[2] Testing of detection tools for AI-generated text,	2023	Empirical comparison of 14 text detectors	Human and generated text with transformations	Reliability and obfuscation weaknesses observed	Natural-language evaluation, not Python code	Supports cautious educational use
[3] Comparing techniques for authorship attribution of source code,	2014	N-grams, software metrics, ranking and classifiers	Multiple language/author collections	Common evaluation collections enable meaningful method comparison	Human author attribution differs from human/AI classification	Motivates lexical and structural baselines
[4] Random Forests,	2001	Randomized tree ensemble	General classification benchmarks	Combines tree strength and diversity	Foundational algorithm, not code detection study	Basis of implemented ensemble

Indexing verification gap: four journal publications have verified publisher records. AJIC explicitly lists Web of Science via SciELO Citation Index; current authoritative Scopus/WoS records for all other venues were not established here. This table must not be represented as a verified 20–25-article indexed literature survey.